BDLT

Blockchain & Distributed Ledger Technology

Viva Preparation Guide — All Experiments

Covers: Fundamentals + Experiment Flow + Viva Q&A

PART 1

Core Fundamentals — Know These Before the Viva

Core Concepts You Must Know

1. What is Blockchain?

Simple Definition

A blockchain is a chain of blocks where each block stores data (like transactions). Every block contains the hash of the previous block, making it impossible to alter past records without changing all subsequent blocks. It is distributed — meaning thousands of computers hold a copy of the same chain simultaneously.

Key Properties:

- Decentralized — no single authority controls it
- Immutable — once written, data cannot be changed
- Transparent — anyone can read the chain (in public blockchains)
- Trustless — transactions are verified by code (consensus), not by a person

A block contains:

- Index — position in the chain
- Timestamp — when it was created
- Data — transactions or information
- Previous Hash — hash of the block before it (this is what links the chain)
- Hash — fingerprint of this block's own data
- Nonce — a number used during mining (Proof of Work)

2. What is a Hash? (SHA-256)

What is Hashing?

A hash is a fixed-length string produced by passing any data through a mathematical function (SHA-256 in Bitcoin/Ethereum). 'Purva' always gives the exact same hash. Changing even one character completely changes the output. It is a one-way function — you cannot reverse a hash to get the original data.

Why is hashing important in blockchain?

- Every block has a hash. Change any data in a block → its hash changes → the next block's 'previous hash' no longer matches → the entire chain breaks
- This is why tampering is detectable — one change breaks all subsequent blocks

3. What is Proof of Work (Mining)?

Proof of Work Explained

Mining is the process of finding a special nonce (a number) such that the block's hash starts with a required number of zeros (the 'difficulty'). The computer keeps incrementing the nonce and recalculating the hash until it finds a matching one. This requires real computational effort — making it costly to tamper with blocks.

In the Python experiment:

- difficulty = 4 means the hash must start with '0000'
- The loop tries nonce = 0, 1, 2, 3... until the condition is met
- This is called 'mining' the block

4. What is a Smart Contract?

Smart Contract Definition

A smart contract is a program stored on the blockchain that automatically executes when predefined conditions are met. No middleman is needed. Once deployed, it cannot be changed. Anyone can verify the code. It runs on the Ethereum Virtual Machine (EVM).

- Written in Solidity (a programming language for Ethereum)
- Compiled into bytecode that the EVM understands
- Deployed at a specific address on the blockchain
- Functions can be called by anyone (public) or restricted (onlyAdmin)

5. Solidity Key Terms

Term	Meaning
`pragma solidity`	Specifies which version of Solidity compiler to use
`contract`	Like a class in programming — defines the smart contract
`uint`	Unsigned integer — a non-negative number
`address`	A 20-byte Ethereum wallet or contract address
`mapping`	Key-value store, like a dictionary/hashmap
`msg.sender`	The address that called the current function
`msg.value`	Amount of ETH (in wei) sent with the function call

`public`	Visible and callable from outside the contract
`view`	Function reads data but does NOT modify the blockchain
`payable`	Function can receive ETH
`modifier`	Reusable condition checker (like onlyAdmin)
`require()`	Checks a condition; reverts the transaction if false
`constructor()`	Runs once when contract is deployed
`struct`	Custom data type grouping multiple fields
`emit / event`	Logs an event on the blockchain (like a notification)

6. What is Ganache?

Ganache — Your Local Fake Blockchain

Ganache is a local blockchain simulator for development. It runs on your computer and gives you 10 test accounts, each pre-loaded with 100 fake ETH. It is not connected to the real Ethereum network — nothing costs real money. It runs on port 7545 by default.

Why use Ganache?

- Test contracts without spending real ETH
- Instant transactions — no waiting for mining
- See all accounts, balances, and transactions in a nice UI
- Works offline — no internet needed

7. What is Truffle?

Truffle — The Developer's Toolkit

Truffle is a development framework for Ethereum. It provides: a project structure (contracts/, migrations/, test/), a compiler that converts Solidity into bytecode, a migration system to deploy contracts, and an interactive console to test deployed contracts.

Key Truffle commands:

- truffle init — creates the project folder structure
- truffle compile — compiles .sol files into ABI + bytecode
- truffle migrate --reset — deploys contracts to the blockchain
- truffle console — opens an interactive JS prompt to call contract functions

8. What is Remix IDE?

Remix IDE — Browser-Based Solidity Editor

Remix IDE (remix.ethereum.org) is a browser-based development environment for Solidity. You can write, compile, and deploy Solidity contracts directly in your browser without installing anything. It has a built-in virtual machine (Remix VM) for testing and can connect to MetaMask for real testnet deployment.

- Solidity Compiler tab — compiles your .sol file
- Deploy & Run tab — deploys the contract and shows interactive buttons for all functions
- Remix VM (Cancun) — local fake blockchain inside the browser, perfect for testing
- Injected Provider - MetaMask — connects to MetaMask for Sepolia testnet deployment

9. What is MetaMask?

MetaMask — Your Ethereum Wallet in the Browser

MetaMask is a browser extension that acts as an Ethereum wallet. It stores your private keys, lets you connect to different networks (Mainnet, Sepolia testnet, local Ganache), signs transactions on your behalf, and acts as a bridge between Remix IDE and the Ethereum network.

- 12-word Secret Recovery Phrase — if lost, wallet is unrecoverable
- Wallet address starts with 0x (20 bytes = 40 hex chars)
- Sepolia — Ethereum testnet where you use free test ETH from a 'faucet'
- Faucet — a website that gives you free test ETH for development

10. What is Ethers.js and Web3.js?

Ethers.js / Web3.js — Talk to Blockchain from JavaScript

These are JavaScript libraries that let your Node.js program interact with the Ethereum blockchain. You can check wallet balances, read block numbers, call contract functions, and send transactions — all from regular JavaScript code.

- ethers.JsonRpcProvider — connects to the blockchain via an RPC URL (e.g., Infura, Alchemy)
- provider.getBlockNumber() — reads the latest block number
- provider.getBalance(address) — gets ETH balance of a wallet
- ethers.formatEther(balance) — converts wei to ETH (1 ETH = 10^18 wei)

11. What is Hyperledger Fabric?

Hyperledger Fabric — Blockchain for Businesses

Hyperledger Fabric is a private/permissioned blockchain platform by the Linux Foundation. Unlike Ethereum (public), Fabric is used by enterprises where participants are known and

trusted. It uses 'chaincode' (smart contracts written in Go, Java, or JavaScript) instead of Solidity.

Key Hyperledger Fabric terms:

- Channel — a private subnet within the network where specific participants transact
- Peer — a node that maintains the ledger and runs chaincode
- Orderer — orders transactions and distributes blocks to peers
- Chaincode — smart contract deployed on Fabric (equivalent to Solidity contract)
- Ledger — the database of all transactions on the channel
- MSP (Membership Service Provider) — manages identities and certificates
- invoke — writes data to the ledger (costs a transaction)
- query — reads data from the ledger (no transaction needed)

12. What is a Token / Cryptocurrency?

Tokens on Ethereum

A token is a unit of value created by a smart contract on a blockchain. ERC-20 is the standard interface for fungible tokens on Ethereum. The token contract keeps a mapping of address → balance. The constructor mints all tokens to the deployer. The transfer() function moves tokens between addresses.

- name — token name (e.g., PurvaCoin)
- symbol — short ticker (e.g., PVC)
- decimals — usually 18, meaning 1 token = 10^18 smallest units
- totalSupply — total tokens ever created
- balanceOf(address) — how many tokens a wallet holds
- transfer(to, amount) — moves tokens from caller to another address

13. What is an NFT?

Non-Fungible Token (NFT)

An NFT is a unique token on the blockchain — unlike regular tokens where each one is identical, each NFT has a unique ID. ERC-721 is the Ethereum standard for NFTs. Common use: digital art, certificates, collectibles. Each NFT has a tokenId and a tokenURI that points to the metadata (description, image).

- mint() — creates a new NFT and assigns it to an address
- tokenURI — a URL pointing to the NFT's metadata (usually stored on IPFS)
- ownerOf(tokenId) — returns who owns a specific NFT

PART 2

Experiment-by-Experiment: Flow, Reasoning & Viva Q&A

Experiment 1 — Truffle + Ganache: Deploy SimpleStorage

What is the Aim?

Install Truffle, connect it to Ganache (local blockchain), write a SimpleStorage Solidity contract, compile it, and deploy it. Then call set() and get() functions to store and retrieve a number.

Experiment Flow (Step-by-Step Reasoning)

1	Install Node.js — Truffle is a Node.js tool; npm is needed to install it
2	npm install -g truffle — installs Truffle globally so 'truffle' command works everywhere
3	Open Ganache → Quickstart — starts 10 pre-funded test accounts on port 7545
4	mkdir + truffle init — creates project folders: contracts/, migrations/, truffle-config.js
5	Edit truffle-config.js — tells Truffle where the blockchain is (127.0.0.1:7545, network_id: '*')
6	Write SimpleStorage.sol — a contract with a uint variable, set() to store it, get() to read it
7	Write 2_deploy_contracts.js — tells Truffle WHICH contract to deploy (migration file)
8	truffle compile — Solidity code → ABI (interface) + bytecode (EVM machine code)
9	truffle migrate --reset — sends bytecode to Ganache; creates a transaction; contract gets an address
10	truffle console → call set(10) then get() — sends transactions to the deployed contract; get() returns '10'

Viva Q&A

Viva Question	Answer
What is Ganache?	A local blockchain simulator for development. Gives 10 test accounts with 100 fake ETH each. Runs on port 7545.
What is Truffle?	A development framework providing compiler (truffle compile), deployer (truffle migrate), and console to test contracts.

What is truffle init?	Creates the project scaffold — contracts/, migrations/, and truffle-config.js
What is truffle compile?	Converts .sol Solidity files into ABI and bytecode that can be deployed to the EVM
What is ABI?	Application Binary Interface — the interface description of all functions and their parameters. Frontend uses ABI to call contract functions.
What is truffle migrate?	Runs the migration files to deploy contracts to the blockchain. --reset redeploys from scratch.
What does the migration file do?	It tells Truffle which contract to deploy. artifacts.require() loads the compiled contract; deployer.deploy() deploys it.
What is the purpose of SimpleStorage?	To demonstrate basic smart contract storage — a uint variable that can be set and retrieved on-chain.
Why is 127.0.0.1:7545 used?	Because Ganache runs locally (loopback address) and its RPC server listens on port 7545 by default.
What does network_id: '*' mean?	Accept any network ID — so the config works with any version of Ganache without specifying an exact ID.

Experiment 2 — Calculator Contract (Add Numbers on Blockchain)

What is the Aim?

Deploy a Calculator smart contract on the local Ganache blockchain using Truffle. Call add(5, 3) and verify the result is 8. This shows that arithmetic can happen on-chain.

Experiment Flow

1	Start Ganache (same as Exp 1) — local blockchain must be running
2	Create Calculator.sol — has uint result, function add(a,b) and getResult()
3	Update migration file to deploy Calculator instead of SimpleStorage
4	Delete build/ folder — avoids stale compiled artifacts from previous experiment
5	truffle compile — compiles Calculator.sol
6	truffle migrate --reset — deploys Calculator to Ganache
7	truffle console — open interactive session
8	let instance = await Calculator.deployed() — gets reference to the deployed contract
9	await instance.add(5, 3) — sends a transaction; contract stores result = 8

10	res = await instance.getResult(); res.toString() → '8' — reads the result

Viva Q&A

Viva Question	Answer
Why do we use 'await' in the console?	Because all blockchain calls are asynchronous — they involve network communication. await waits for the transaction to complete.
What is the difference between add() and getResult()?	add() is a state-changing function — it creates a transaction and costs gas. getResult() is a view function — it only reads data and costs no gas.
Why delete the build/ folder?	The build/ folder has compiled artifacts from the previous experiment. Deleting it ensures a clean compile for the new contract.
What is wei? What is ETH?	Wei is the smallest unit of Ether. 1 ETH = 10^18 wei. All values inside Solidity are in wei.
What does 'public' keyword mean?	The function is visible and callable from outside the contract by any address.
Why store result as state variable?	Solidity functions can't return values from transactions — state variables persist the result on-chain so getResult() can read it later.

Experiment 3 — MetaMask + Ethers.js (Web3 Program)

What is the Aim?

Install MetaMask as a browser extension, create a wallet, connect to Sepolia testnet, get test ETH from a faucet. Then write a Node.js program using Ethers.js to connect to Sepolia, read the current block number, and check a wallet's ETH balance.

Experiment Flow

1	Install MetaMask Chrome extension — creates a local wallet (private key stored in browser)
2	Create wallet → save 12-word recovery phrase — this phrase IS the wallet; guard it carefully
3	Switch to Sepolia Test Network — Ethereum testnet using fake ETH
4	Copy wallet address (0x...) — needed to receive test ETH from faucet

5	Go to Sepolia faucet → paste address → receive test ETH — now wallet has funds for testing
6	Create web3-project folder → npm init -y → npm install ethers — sets up Node.js project
7	Create app.js — write JS code using ethers.JsonRpcProvider with an Infura/Alchemy RPC URL
8	provider.getBlockNumber() — connects to Sepolia and reads latest block
9	provider.getBalance(address) → ethers.formatEther() — reads balance in wei, converts to ETH
10	node app.js — runs the program; output shows block number and wallet balance

Viva Q&A

Viva Question	Answer
What is MetaMask?	A browser extension wallet that stores your Ethereum private key, lets you sign transactions, and connects web apps to the blockchain.
What is a testnet?	A test version of Ethereum that uses fake ETH. Sepolia is Ethereum's primary testnet. Used for development without spending real money.
What is a faucet?	A website that gives free test ETH for development purposes. You paste your wallet address and receive test tokens.
What is Ethers.js?	A JavaScript library for interacting with Ethereum — reading balances, calling contracts, sending transactions.
What is a JsonRpcProvider?	A connection to the Ethereum network through an RPC (Remote Procedure Call) endpoint like Infura or Alchemy. It's how your JS app talks to the blockchain.
What is formatEther()?	Converts a balance from wei (10^18 per ETH) to a human-readable ETH string. E.g., 1000000000000000000 wei → '1.0' ETH
What is a recovery phrase?	A 12-word mnemonic that is the master key to your wallet. Anyone with these words can access all your funds.
What is a private key vs. public key?	Private key is secret — used to sign transactions. Public key (wallet address) is shared — others use it to send you ETH.

Experiment 4 — Remix + MetaMask + Web3.js on Sepolia Testnet

What is the Aim?

Write a SimpleStorage contract in Remix IDE, compile it, deploy it to the Sepolia testnet using MetaMask, interact with it (set and get data), and then write a Web3.js program to connect to Sepolia and read the current block number.

Experiment Flow

1	Setup MetaMask on Sepolia + get test ETH (same as Exp 3)
2	Open remix.ethereum.org — browser-based IDE, no installation needed
3	Create SimpleStorage.sol — same contract: setData(uint) and getData()
4	Solidity Compiler tab → select 0.8.0+ → Compile — converts code to ABI + bytecode
5	Deploy & Run tab → Environment: Injected Provider - MetaMask — connects Remix to your MetaMask wallet
6	Click Deploy → MetaMask popup appears → Confirm — sends the contract deployment transaction to Sepolia
7	Contract appears under 'Deployed Contracts' with its Sepolia address
8	Call setData(100) → Confirm in MetaMask — stores 100 on Sepolia blockchain (real testnet transaction)
9	Click getData() → Output: 100 — reads data back from the deployed contract
10	Create Node project → npm install web3@1.10.0 → write app.js with Web3 to read block number

Viva Q&A

Viva Question	Answer
What is Remix IDE?	A browser-based Solidity development environment. Write, compile, and deploy contracts without installing anything locally.
What is 'Injected Provider - MetaMask'?	It means Remix uses MetaMask as the connection to the blockchain. Remix sends transactions through MetaMask which signs and broadcasts them.
What is the difference between Remix VM and Injected Provider?	Remix VM is a local simulated blockchain in the browser (no real network). Injected Provider connects to a real network (like Sepolia) via MetaMask.
What happens when you click Deploy?	Remix compiles the bytecode, creates a deployment transaction, sends it to MetaMask for signing, MetaMask broadcasts it to Sepolia, and the contract gets a unique address.
What is the difference between Ethers.js and Web3.js?	Both do the same job — interact with Ethereum from JavaScript. Ethers.js is newer and simpler. Web3.js is older

	but widely used. The experiment shows Web3.js for variety.
What is a contract address?	When a contract is deployed, it gets a permanent Ethereum address (like 0xABC123...). Anyone who knows this address can interact with the contract.
What is gas?	The computational fee for every operation on Ethereum. Deploying a contract and calling state-changing functions cost gas (paid in ETH). View functions are free.

Experiment 5 — Design and Develop Cryptocurrency (MyToken)

What is the Aim?

Create a custom ERC-20-like cryptocurrency called PurvaCoin (PVC) using Solidity, deploy it on Sepolia via Remix + MetaMask, and demonstrate token transfer between two accounts.

Experiment Flow

1	Setup MetaMask on Sepolia (same as before)
2	Create MyToken.sol in Remix — defines name, symbol, decimals, totalSupply, balanceOf mapping
3	constructor(uint256 _initialSupply) — when deployed with supply=1000, all 1000 tokens go to the deployer
4	transfer(address _to, uint256 _value) — moves tokens from msg.sender to another address with balance checks
5	Compile → Deploy with _initialSupply = 1000 → Confirm in MetaMask
6	Check totalSupply → output: 1000
7	Check balanceOf(Account1 address) → output: 1000 (deployer owns all tokens)
8	Create Account2 in MetaMask
9	Call transfer(Account2 address, 100) → Confirm — 100 tokens move from Account1 to Account2
10	Check balanceOf(Account1) → 900; balanceOf(Account2) → 100 — transfer verified

Viva Q&A

Viva Question	Answer

What is ERC-20?	A token standard for Ethereum — defines name, symbol, decimals, totalSupply, balanceOf, and transfer functions. PurvaCoin is a simplified ERC-20.
What is the mapping in the token contract?	mapping(address => uint256) public balanceOf — stores how many tokens each wallet address holds. Like a giant ledger.
Why is decimals = 18?	To allow fractional tokens. If decimals=18, one token is represented as 10^18 internal units. This is the standard for ERC-20 tokens.
What does the constructor do?	Runs once at deployment. Sets totalSupply and gives all tokens to msg.sender (the deployer's address).
What does require() do in transfer()?	Checks that the sender has enough tokens. If not, the transaction reverts (cancelled) and the state is unchanged.
What is msg.sender?	The Ethereum address that called the current function. In transfer(), it is the person sending tokens.
Difference between token and ETH?	ETH is the native currency of Ethereum. Tokens are created by smart contracts and have no intrinsic value unless the community gives them value.
What is a mapping?	A data structure in Solidity like a dictionary/hashmap. mapping(address => uint256) maps each address to a number (their token balance).

Experiment 6 — Simulate a Basic Blockchain in Python

What is the Aim?

Build a blockchain from scratch in Python to understand how blocks, hashing, hash chaining, and Proof of Work actually work under the hood — without any external tools.

Experiment Flow

1	Create Block class — holds index, timestamp, data, previous_hash, nonce, hash
2	calculate_hash() method — combines all block fields and runs hashlib.sha256() on them
3	mine_block(difficulty) — loops: increment nonce → recalculate hash → check if hash starts with '0000'; stop when found
4	Create Blockchain class — starts with genesis block (index=0, previous_hash='0')
5	create_genesis_block() — the first block; has no real previous hash so it's '0'
6	add_block(new_block) — sets new block's previous_hash = last block's hash → then mines it → appends to chain

7	Add Block 1: 'Purva sends 10 coins to A' — gets mined with difficulty 4
8	Add Block 2: 'A sends 5 coins to B' — links to Block 1's hash
9	Print all blocks — shows index, timestamp, data, previous hash, hash, nonce

Why This Experiment Matters

This is the most fundamental experiment — it shows you exactly HOW a blockchain works without any framework. You can see: (1) Each block's hash depends on its data + previous hash. (2) Mining requires computational work. (3) Changing Block 1's data would change its hash, breaking Block 2's 'previous_hash' link — this is why blockchains are tamper-evident.

Viva Q&A

Viva Question	Answer
What is the genesis block?	The first block in a blockchain. It has no previous block so its previous_hash is set to '0' or a hardcoded string. Every blockchain starts with a genesis block.
What is a nonce?	A number that miners increment until the block's hash meets the difficulty requirement. It has no other meaning — it's just a counter.
What is difficulty?	The number of leading zeros required in the hash. difficulty=4 means hash must start with '0000'. Higher difficulty = more computation needed.
Why does changing data in Block 1 break the chain?	Changing data changes Block 1's hash. Block 2 stores Block 1's old hash as previous_hash — now they don't match, so the chain is broken (tamper detected).
What is hashlib.sha256?	Python's built-in SHA-256 hashing function. Takes any string → returns a fixed 64-character hex string. Same input always gives same output.
What is hash chaining?	Each block stores the hash of the previous block. This links them into a chain. If any block is altered, all subsequent hashes become invalid.
What does encode() do in the hash function?	SHA-256 requires bytes, not a string. .encode() converts the Python string to bytes before hashing.
What is the purpose of timestamp in a block?	Records when the block was created. Also contributes to the hash, so two blocks with identical data but different times will have different hashes.

Experiment 7 — Hyperledger Fabric Chaincode (StudyChain)

What is the Aim?

Write and deploy a chaincode (smart contract) on Hyperledger Fabric's test-network. The chaincode manages student records — add, retrieve, update, and delete. Demonstrates enterprise/permissioned blockchain.

Experiment Flow

1	Install prerequisites — Node.js v18, Docker, Docker Compose, Go, Git (all required by Fabric)
2	Download Hyperledger Fabric samples — includes test-network scripts and pre-configured peer nodes
3	Add Fabric binaries to PATH — so 'peer' command works from terminal
4	./network.sh up createChannel -c mychannel -ca — starts Docker containers: 2 peers, 1 orderer, creates 'mychannel'
5	Write studychain.js — extends Contract class; implements initLedger, addStudent, getStudent, updateCourse, deleteStudent
6	ctx.stub.putState(key, value) — writes data to the ledger; ctx.stub.getState(key) — reads from ledger
7	./network.sh deployCC — packages, installs, approves, and commits chaincode to mychannel
8	Set environment variables (CORE_PEER_*) — tells peer CLI which peer to connect to and with which identity
9	peer chaincode invoke — writes to ledger: initLedger, addStudent('S101','Purva','BTech'), updateCourse
10	peer chaincode query — reads from ledger: getStudent('S101') returns JSON record

Viva Q&A

Viva Question	Answer
Difference between Hyperledger Fabric and Ethereum?	Ethereum is public — anyone can join. Fabric is permissioned — only known participants with certificates can join. Fabric is for enterprise use; Ethereum is general purpose.
What is a channel in Fabric?	A private communication subnet within the network. Only channel members can see transactions on that channel.
What is chaincode?	Smart contracts in Hyperledger Fabric. Written in JavaScript, Go, or Java. Defines business logic for reading/writing ledger data.

Difference between invoke and query?	Invoke modifies the ledger (creates a transaction, requires ordering and consensus). Query reads from ledger (no transaction, instant result).
What is ctx.stub?	The chaincode stub — provides API methods to interact with the ledger: putState (write), getState (read), deleteState (delete).
What is Docker's role?	Fabric runs as Docker containers. Each peer, orderer, and CA is a separate Docker container. Docker provides the isolated environment for each component.
What is an orderer?	A Fabric component that collects transactions from peers, orders them into blocks, and distributes blocks to all peers. It does not execute chaincode.
What is MSP?	Membership Service Provider — manages identities (certificates) in Fabric. CORE_PEER_LOCALMSPID identifies which organization the peer belongs to.
Why is Go installed?	Hyperledger Fabric's core tools (like the peer binary) are written in Go and need Go to compile or run certain operations.

Experiment 8 Series — Smart Contract Mini-Projects

All Experiment 8 mini-projects follow the same deployment pattern in Remix IDE (Remix VM or Sepolia). The key differences are in the smart contract logic.

8.1 — Academic Certificate Verification

What it does
University admin issues tamper-proof digital certificates stored on blockchain. Anyone can verify authenticity by passing the certificate hash. No middleman needed.

Key design decisions:

- onlyAdmin modifier — only the university (contract deployer) can issue certificates
- mapping(string => Certificate) — certificate hash is the unique key; prevents duplicates
- certHash as key — a hash of the certificate document itself; if document is tampered, hash changes and lookup fails
- issueBatch() — allows issuing multiple certificates in one transaction (gas efficient for graduation ceremonies)
- verifyCertificate() is view — public verification costs no gas

Viva Question	Answer

Why use a hash as key instead of studentId?	Multiple students could have the same ID format. The certificate hash is unique per document — ensures no duplicates.
What does 'exists: bool' field do?	Acts as a sentinel value. Mapping returns default (empty struct) for unknown keys — the 'exists' flag distinguishes a real entry from a missing one.
Why can't the admin change a certificate once issued?	Blockchain is immutable. Once a transaction is confirmed, the data is permanent. This is actually the key benefit — certificates can't be faked or altered.
What is the certHash in practice?	A SHA-256 or keccak256 hash of the actual certificate PDF/data. Employers hash the certificate they received and compare it with the blockchain entry.

8.2 — Decentralized Voting System

What it does
Admin deploys the contract, adds candidates, starts voting. Each Ethereum address can vote exactly once. Admin ends voting. Results are transparent and tamper-proof.

- hasVoted mapping — prevents double voting; msg.sender is the unique voter ID
- votingActive bool — voting only works between startVoting() and endVoting() calls
- candidates[] array — stores Candidate structs with name and voteCount
- onlyDuringVoting modifier — vote() reverts if voting is not active

Viva Question	Answer
How does it prevent double voting?	mapping(address => bool) hasVoted. When a user votes, hasVoted[msg.sender] = true. The require(!hasVoted[msg.sender]) check blocks a second vote from the same address.
Why is voting more secure on blockchain?	Votes are immutable once cast. The counting logic is in code (auditable). No single authority can manipulate results. Results are publicly verifiable.
What is a modifier?	A reusable function wrapper. onlyAdmin checks msg.sender == admin before running the function. onlyDuringVoting checks votingActive == true.

8.3 — Crowdfunding Platform

What it does
Anyone can create a fundraising campaign with a title, goal (in wei), and deadline. Others donate ETH. If goal is reached by deadline, creator claims funds. If not, donors get a refund.

- payable functions — donate() and constructor accept ETH
- block.timestamp — Solidity's way to check current time; campaign.deadline is set as block.timestamp + duration
- msg.value — the amount of ETH sent with the function call
- (bool sent,) = payable(creator).call{value: amount}('') — secure way to send ETH from contract to an address
- Goal not reached + deadline passed → refund() available; goal reached → claimFunds() available

Viva Question	Answer
What is block.timestamp?	A global variable in Solidity giving the Unix timestamp of the current block. Used to check deadlines.
Why is 1 ETH represented as 1000000000000000000 in the goal?	Solidity works in wei. 1 ETH = 10^18 wei = 1000000000000000000. Goals must be set in wei.
What is the reentrancy concern in claimFunds?	We set fundsClaimed = true BEFORE sending ETH. This prevents a malicious contract from calling claimFunds() again during the ETH transfer (reentrancy attack).

8.4 — Decentralized To-Do List

What it does
Each user has their own private task list on the blockchain. Tasks can be added, updated, and marked completed. No one else can access or modify another user's tasks.

- mapping(address => Task[]) userTasks — each address has its own array of tasks
- msg.sender as key — only the caller can access their own tasks
- Tasks are private but code is public — anyone can read the contract code but data access is controlled by msg.sender

8.5 — Simple Bank (Wallet System)

What it does
A smart contract bank. Users deposit ETH, it's held by the contract. Users can withdraw their own balance. No one can withdraw another user's funds.

- mapping(address => uint256) private balances — each user's balance, private to the contract
- deposit() is payable — msg.value ETH is added to the user's balance
- withdraw() checks balance first (require), deducts balance, then sends ETH
- Contract holds ETH — the smart contract's address holds the deposited ETH

Viva Question	Answer
Where does the ETH go when you deposit?	It goes to the smart contract's address on the blockchain. The contract holds it until you withdraw.
Why deduct balance before sending ETH?	To prevent reentrancy attacks. If a malicious contract calls withdraw() again during the ETH transfer, the balance is already 0 so it can't withdraw again.

8.6 — Supply Chain Management

What it does

Tracks a product through the supply chain — manufacturer creates it, each participant (shipper, retailer) updates its status. Full history is recorded on the blockchain.

- Each product has a unique ID, current owner, and status (Created, Shipped, Delivered)
- Only the current owner can update the product status (ownership transfer)
- Blockchain provides immutable audit trail — full provenance of the product

8.7 — NFT Minting Contract

What it does

Implements a basic Non-Fungible Token (NFT) contract. Each minted NFT has a unique tokenId and a tokenURI (link to metadata). Only the owner of an NFT can transfer it.

- tokenIdCounter — auto-increments to give each NFT a unique ID
- tokenURI — string URL pointing to JSON metadata (usually hosted on IPFS)
- ownerOf(tokenId) — who owns that specific NFT

8.8 — Subscription Service

What it does

Users pay ETH to subscribe for a set duration. Contract tracks subscription expiry. isSubscribed() returns true only if payment was made and subscription hasn't expired.

- subscriptions mapping stores expiry timestamps per address
- block.timestamp + duration = subscription end time
- isSubscribed checks block.timestamp < expiry time

8.9 — Escrow Payment System

What it does
A buyer deposits ETH into the escrow contract. Funds are released to the seller only when the buyer confirms delivery. If cancelled, buyer gets a refund.

- State machine: Awaiting Delivery → Complete or Refunded
- Only the buyer can confirm delivery or request refund
- Contract holds ETH until confirmation — neither party can access it unilaterally

8.10 — Decentralized Lottery

What it does
Players pay ETH to enter the lottery. Owner draws the winner — a pseudo-random player is selected using block data. Winner gets the entire pot.

- players[] array stores all participant addresses
- Pseudo-random selection: keccak256(block.timestamp, block.difficulty, players.length) % players.length
- Note: not truly random — on-chain randomness is a known limitation; Chainlink VRF is the production solution

Viva Question	Answer
Why is on-chain randomness a problem?	Miners/validators can predict block.timestamp and block.difficulty. A malicious miner could manipulate these to make themselves win. True randomness requires an oracle like Chainlink VRF.

8.11 — Certificate Issuance and Verification

What it does
Similar to 8.1 but focused on a simpler, single-issuance model. Demonstrates the core certificate storage and verification pattern — issuer stores certificate hash, verifier checks it.

PART 3

Quick Revision — Must-Know One-Liners for Viva

Quick Revision Table

Term	One-Line Definition
Blockchain	Chain of blocks where each block's hash depends on previous — making data tamper-evident
Block	A unit in the chain containing index, timestamp, data, previous hash, hash, and nonce
Hash	A fixed-length fingerprint of data. Same input = same hash. One-way function.
Nonce	A number miners increment until the hash meets the required difficulty (starts with N zeros)
Proof of Work	Finding a nonce such that hash starts with a required number of zeros — requires computational effort
Genesis Block	The very first block; has no previous block so previous_hash = '0'
Smart Contract	Self-executing code on the blockchain; runs automatically when conditions are met; cannot be changed after deployment
Solidity	The programming language used to write Ethereum smart contracts
EVM	Ethereum Virtual Machine — the runtime that executes smart contract bytecode on all Ethereum nodes
ABI	Application Binary Interface — describes all contract functions and parameters; used by frontend to call contracts
Bytecode	Machine code for the EVM produced by compiling Solidity; what actually gets deployed on-chain
Ganache	Local blockchain simulator for development; 10 test accounts with 100 fake ETH each; runs on port 7545
Truffle	Ethereum development framework: compile, migrate (deploy), console
truffle migrate	Deploys compiled contracts to the blockchain using migration files
Remix IDE	Browser-based Solidity editor with compiler and deployer built in
MetaMask	Browser wallet extension that stores private keys and signs blockchain transactions
Sepolia	Ethereum's test network — uses fake ETH; for development before mainnet
Faucet	Website that gives free test ETH for Sepolia or other testnets
Ethers.js	JavaScript library to interact with Ethereum — read balances, call contracts
Wei	Smallest unit of Ether. 1 ETH = 10^18 wei
msg.sender	The Ethereum address that called the current smart contract function

msg.value	Amount of ETH (in wei) sent with a payable function call
payable	Keyword allowing a function or address to receive ETH
view function	Reads blockchain data without creating a transaction — free to call
require()	Solidity condition check — if false, transaction reverts and no state changes occur
mapping	Solidity key-value store. mapping(address => uint256) maps each address to a number
modifier	Reusable function guard in Solidity (e.g., onlyAdmin checks msg.sender == admin)
Hyperledger Fabric	Permissioned/private enterprise blockchain — participants are known and verified
Chaincode	Smart contracts in Hyperledger Fabric (written in JS, Go, or Java)
Channel (Fabric)	Private subnet in Fabric where only channel members see transactions
peer invoke	Fabric command to write data to the ledger (creates a transaction)
peer query	Fabric command to read data from ledger — no transaction created
ctx.stub.putState	Fabric chaincode API to write a key-value pair to the ledger
ERC-20	Ethereum token standard — defines name, symbol, decimals, totalSupply, balanceOf, transfer
ERC-721	Ethereum NFT standard — each token has a unique ID (tokenId) unlike fungible ERC-20
Escrow	ETH held by a smart contract until a condition (like delivery confirmation) is met
Reentrancy	Attack where malicious contract calls back into a function before it completes — prevented by updating state before sending ETH
Immutability	Once deployed, contract code cannot be changed. Once a block is mined, its data is permanent.
Decentralized	No single point of control — thousands of nodes hold a copy of the ledger

Good luck with your Viva!

Remember: Understand the WHY behind each step — that's what viva examiners want to hear.